

Práctica 1

Sitios web estáticos y dinámicos y Servlets/Alcances/Listeners

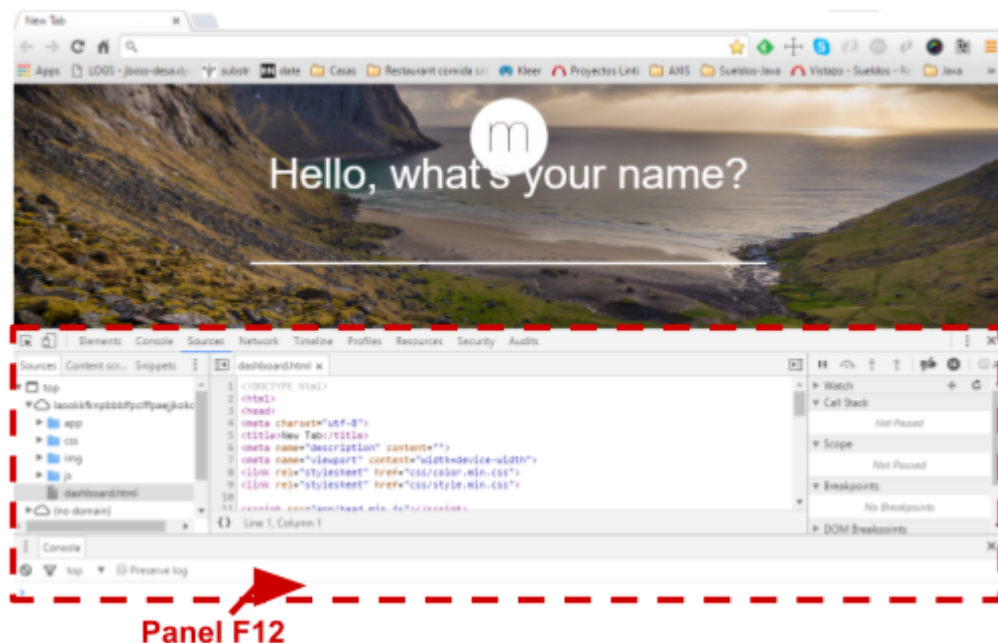
Como entorno de desarrollo se utilizará alguno de los siguientes:

- Eclipse IDE for Enterprise Java and Web Developers (<https://www.eclipse.org/downloads/packages/>)
- IntelliJ Ultimate (<https://www.jetbrains.com/idea/download/>)

1. Conceptos básicos.

Para este punto usará el navegador Chrome o Mozilla Firefox y analizará los intercambios de mensajes HTTP entre el navegador y los servidores Web.

Usando la tecla F12 se habilitará la herramienta para el desarrollador.



Responda las siguientes preguntas:

- ¿Qué es una URL y cómo se compone?
- ¿Qué componentes de la URL son opcionales?
- ¿Cómo se especifica en el encabezado HTTP el recurso al que se quiere acceder?
- ¿Cómo se diferencian los mensajes de requerimiento HTTP de los mensajes de respuesta?
- ¿Cómo sabe el servidor qué navegador está utilizando el visitante? (Revise la información provista por la herramienta para el desarrollador (F12) respecto de los Headers del Request)
- ¿Cómo se envían los parámetros en los mensajes con método POST?
- ¿Qué son las Cookies? ¿Cómo se envían las Cookies a través de HTTP?
- ¿Cuál es la diferencia entre HTTP y HTTPS? ¿Cuáles son los puertos por default (por defecto) de los mismos?

2. **Códigos de error HTTP.** En función de la información que se encuentra disponible en la siguiente URL, <https://www.w3.org/Protocols/rfc2616/rfc2616-sec10.html>, responda:
- ¿Cómo se **categorizan** los tipos de respuestas HTTP según su código de estado?
 - Complete la siguiente tabla indicando el significado de los códigos de estado

Código	Descripción
200	
201	
204	
206	
301	
302	
304	
401	
403	
404	
409	
500	

- En el browser, escriba la URL: <http://www.eldia.com>, ¿cuáles de estos códigos aparecen?
- Verifique como se redirecciona <http://www.guarani-informatica.unlp.edu.ar> a una conexión segura (use la pestaña *Network* de la herramienta para el desarrollador que brinda el browser, en el caso de usar Chrome, verifique que se encuentra chequeada la opción "Preserve Log"). ¿En qué campo de la respuesta se indica a donde redireccionar?

NOTA: Para los siguientes puntos, no copiar directamente desde el documento PDF al IDE. Copiar del PDF a un archivo .txt y luego del archivo .txt al IDE, para filtrar caracteres que no sean estándar.

5. Web dinámica y Servlets. Sitio web Clasificados

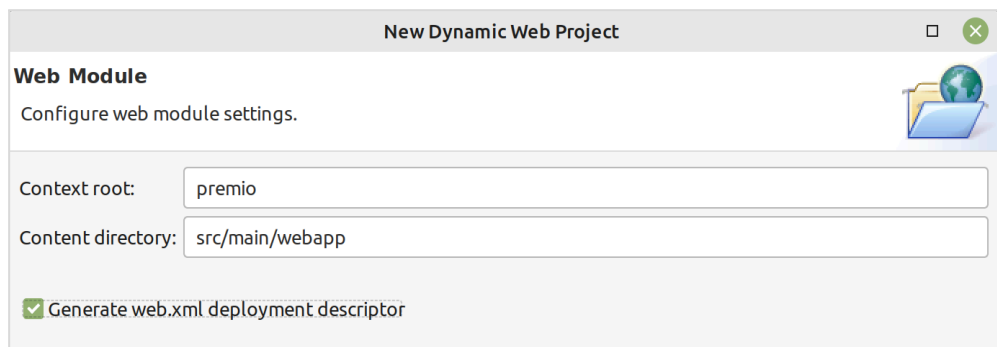
Descargue Tomcat 11 de <https://tomcat.apache.org/download-11.cgi>

Cree un proyecto web dinámico llamado **clasificados**

El sitio web clasificados tendrá una “página de login” que proveerá acceso a dos tipos de perfil: “publicador” ó “administrador”

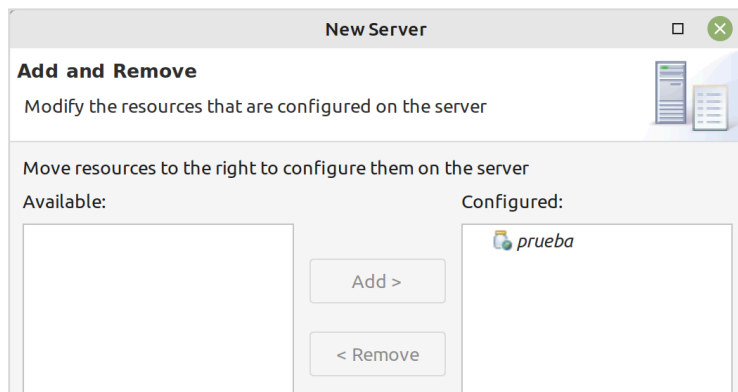
Usando Eclipse

- En el combo **“Dynamic Web module versión”** seleccione la versión **5.0**. Asigne como nombre de la aplicación web (*context root*) el nombre **premio**.
Genere el archivo web.xml tildando **Generate web.xml deployment descriptor** (esta vista aparece luego de clickear dos veces **Next**)



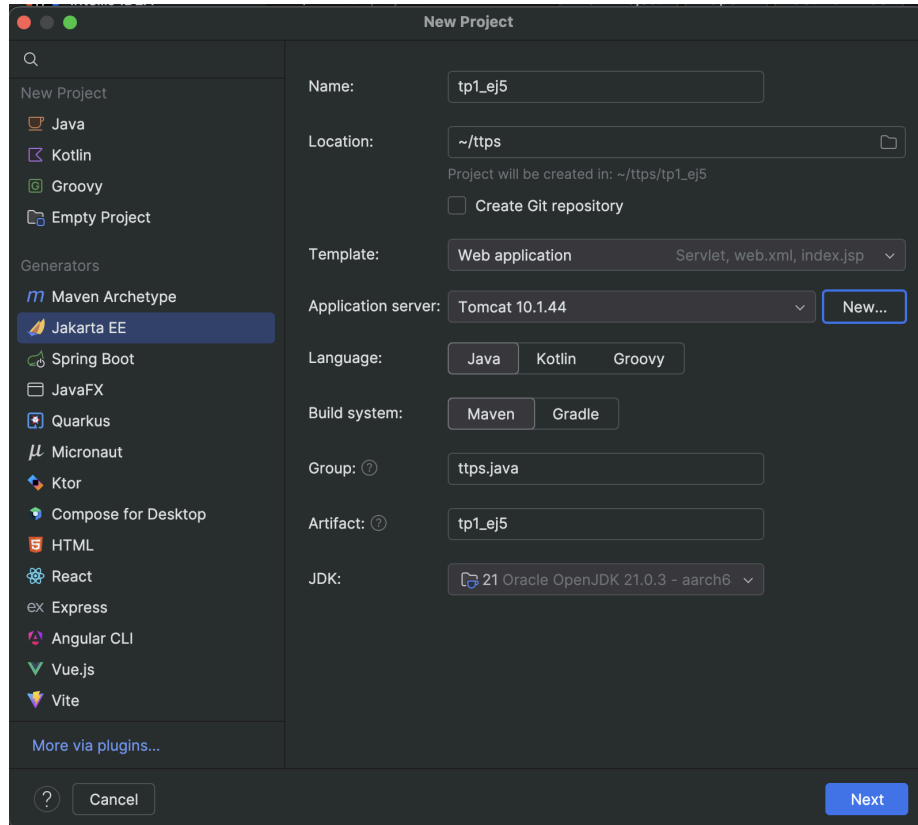
También puede crear el archivo descriptor luego de crear el proyecto con el botón derecho del mouse, usando la opción **Java EE Tools -> Generate Deployment Descriptor Stub**.

- Analice la estructura de directorios que muestra la vista *Project Navigator*.
- Desde la vista *Window - Show view - Servers* agregue un nuevo server. Elija la versión del Apache Tomcat descargado y el directorio de instalación. Asocie su proyecto al Tomcat desde la siguiente vista



Usando IntelliJ

- a. En el IDE seleccione Nuevo Proyecto de tipo “Jakarta EE”, Template “Web application” y agregue el Tomcat descargado como Application Server, indicando la ruta a la carpeta.



- b. En la siguiente pantalla seleccione la versión de Jakarta EE 10.
- c. Analice la estructura de directorios que muestra la vista Project. Elimine el archivo *index.jsp* ya que usaremos html plano.

Usando cualquiera de los IDEs

- d. Escriba una página HTML llamada **login.html**, que contenga al menos:
 - i. un *título*
 - ii. un *formulario* con dos campos de texto con las etiquetas Usuario y Clave donde el usuario ingresará sus datos de acceso y un botón de tipo “*submit*” para enviar los datos ingresados al servlet del inciso b.
- e. Escriba un servlet llamado **Login**, que tome del requerimiento http, los datos de un usuario y los valide usando datos mantenidos por el propio servlet.

Para la validación, el servlet debe definir en su estado una colección de objetos Usuario (un Usuario tiene un nombre de usuario, clave y perfil) contra los cuales

validar. Los usuarios se deben instanciar y guardar en la inicialización del Servlet. Utilice un objeto List para guardar los usuarios y Streams para buscarlos.

Si los datos son válidos: *redirecciona*, usando el método *sendRedirect()*, a una página HTML que contenga un menú de operaciones disponibles de acuerdo al perfil del usuario. Deberá crear una página para el perfil *Publicador* y otra página para el perfil *Administrador*.

perfil	operaciones disponibles
Publicador	Actualizar Datos de Contacto ABM de Publicaciones Contestar Consultas
Administrador	Listar Usuarios Publicadores ABM Administradores Ver Estadísticas

- Si los datos son inválidos: redirecciona a la página **error.html** donde se informa que los datos ingresados no son válidos y se muestra un link a la página **login.html**, para que el usuario vuelva a ingresar sus datos.
- Una vez funcionando, genere el archivo war de la aplicación y realice su despliegue en el Tomcat por fuera del IDE.
- Analice las diferencias de estructura entre el war y el proyecto del IDE.
- Agregue al servlet *Premio* una funcionalidad que permita devolver en formato JSON el último usuario que accedió al sitio y el número de visitas. Puede utilizar el método GET con un parámetro *formato* para indicar el formato JSON.
- ¿Qué **content-type** debería utilizar para dar esta respuesta?
- Para la declaración de un servlet se puede utilizar el archivo web.xml o anotaciones. ¿Cuál de los métodos utilizó en su proyecto? Duplique el proyecto y realice las modificaciones necesarias para tener ambas soluciones funcionando. ¿Qué método tiene prioridad sobre el otro si hubiera declarado el servlet de ambas maneras en un mismo proyecto?

6. sendRedirect() vs. forward()

- Modifique el servlet **Login** para que en vez de redireccionar (con *sendRedirect()*) a una página estática, **delegue el requerimiento y la respuesta a otro servlet** llamado **Menú**.

El servlet **Login** debe enviar **dentro del request** la información del perfil de usuario al servlet **Menú**. El servlet **Menú** mostrará cual es el perfil y generará dinámicamente el menú correspondiente al perfil de usuario.

- ¿Es posible enviar información del perfil desde el servlet Login al servlet Menú usando *sendRedirect()*? De ser posible indique cómo lo implementaría.

7. Listeners

- a. Cree una clase llamada **SitioClasificado** con 3 variables de instancia: nombre, email y teléfono de contacto de tipo String.
- b. Agregue en el archivo WEB-INF/web.xml **tres (3) parámetros de contexto** que representan información de un sitio web: nombre del sitio, email y teléfono de contacto.

Usando Eclipse

Puede crearlo haciendo click con el botón derecho del mouse, usando la opción Java EE Tools -> Generate Deployment Descriptor Stub.

Al crear un nuevo Dynamic Web Project, también se puede seleccionar la opción "crear web.xml".

Usando IntelliJ

Puede crearlo seleccionado File -> Project Structure -> Facets, elige el módulo de tu aplicación web, en Deployment Descriptors hacer clic en el botón + para agregar un nuevo descriptor, seleccionando web.xml para crearlo.

Al crear un nuevo proyecto Jakarta EE, también se puede seleccionar la opción "Crear web.xml".

- c. Implemente un **Listener de Contexto** que lea del archivo **web.xml** los valores agregados en el punto b. Con esos datos cree una instancia de la clase **SitioClasificado**. Esta instancia será luego guardada **en el contexto de la aplicación Web**.

Usando Eclipse

Tendrá la opción en el menú -> New Listener (seleccionar Contextlistener)

Usando IntelliJ

IntelliJ IDEA no proporciona las plantillas predeterminadas para crear servlets, listeners y filters. Sin embargo, permite crear plantillas. Mas info:

<https://www.jetbrains.com/help/idea/creating-and-configuring-web-application-elements.html#elements-templates>

8. Include

- a. Escriba un servlet llamado **Encabezado** que imprima los datos del sitio Web guardados en el punto anterior.
- b. Modifique el servlet **Menú** para que incluya el encabezado generado en el inciso a).

9. Generación de respuestas binarias a partir de Servlets

Se aproxima el Festival Capital en nuestra ciudad y usted ha sido contratado para implementar una aplicación web que permita participar de una promoción que regala camisetas personalizadas. Las camisetas podrán ser retiradas el día del evento en el mismo Hipódromo de La Plata visitando el stand del sponsor oficial TTPS.

- a. Genere una página **index.html** con un campo para ingresar un texto personalizado que se imprimirá en la remera y en el cupón.
- b. Implemente un servlet llamado **ImprimeCupon** que devuelva una imagen dinámica dada con los siguientes ítems:
 - La imagen de la remera del evento (es un archivo remera.jpg que reside en el servidor)
 - el texto personalizado ingresado por el usuario
 - Un código de retiro generado aleatoriamente. Por ej: `ThreadLocalRandom.current().nextInt(1000000,99999999)`

NOTA: Utilice la clase **ImageIO** para leer un archivo JPG y generar una salida JPG escribiendo directamente en la respuesta del Servlet la clase **java.awt.image.BufferedImage** para usar su **Graphics** (**java.awt.Graphics2D**) y dibujar sobre la imagen a devolver.

Por ejemplo si se ingresa la frase "**TTPS presente**" el cupón deberá visualizarse así:

